



УНИВЕРЗИТЕТ У НОВОМ САДУ ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД
Департман за рачунарство и аутоматику
Одсек за рачунарску технику и рачунарске комуникације

Документација пројекта

Кандидат: Маја Миловић
Број индекса: РА69/2024

Предмет: Основе паралелног програмирања и софтверски алати
Тема рада: Преводацац МАН Асемблера у МИПС

Ментор рада: др Теодора Новковић

Нови Сад, јун, 2026.

Sadržaj

1. Uvod.....	1
1.1 MIPS arhitektura	1
1.2 MAVN – MIPS Asembler Visokog Nivoa.....	1
1.3 Zadatak	1
2. Analiza problema	3
3. Koncept rešenja.....	4
3.1 Leksička analiza	4
3.2 Sintaksna analiza	5
3.3 Analiza životnog veka (Liveness Analysis)	5
3.4 Zauzimanje resursa (Alokacija registara)	6
3.5 Generisanje MIPS fajla.....	6
4. Programsko rešenje	7
4.1 Organizacija projekta.....	7
4.2 Leksička analiza	7
4.3 Sintaksna analiza	8
4.4 Analiza životnog veka	8
4.5 Alokacija resursa	9
4.6 Generisanje MIPS .o fajla.....	9
5. Uputstvo za pokretanje projekta	10
6. Verifikacija.....	11
6.1 Testni skupovi	11
6.1.1 simple.mavn – Osnovne instrukcije.....	11
6.1.2 multiply.mavn – Složena alokacija (spill i bez spilla)	12
6.1.3 commands.mavn – Nove instrukcije.....	13
6.1.4 test.mavn – Kompleksno grananje i višestruke labele	14
6.2 Rezultati i analiza	15
7. Zaključak.....	16

SPISAK SLIKA

Slika 1 - Osnovne faze kompajlera (prevodioca)	4
--	---

SPISAK TABELA

Tabela 1 – Proširenje za MAVN assembler – 3 dodatne instrukcije	5
Tabela 2 –Struktura izlazne MIPS datoteke.....	9
Tabela 3 – Prevođenje datoteke <i>simple.mavn</i>	Error! Bookmark not defined. 1-12
Tabela 4 – Prevođenje datoteke <i>multiply.mavn</i>	12-13
Tabela 5 – Prevođenje datoteke <i>commands.mavn</i>	13-14
Tabela 6 – Prevođenje datoteke <i>test.mavn</i>	14-15

1. Uvod

1.1 MIPS arhitektura

MIPS (*Microprocessor without Interlocked Pipeline Stages*) je porodica arhitektura skupa računarskih instrukcija dizajniranih za rad sa MIPS mikroprocesorima. MIPS je RISC arhitektura sa ograničenim brojem registara opšte namene. Programer koji piše direktno na MIPS assembleru mora sam da vodi računa o tome koji su registri trenutno u upotrebi i kakav je njihov sadržaj, što programiranje čini složenijim i podložnijim greškama.

1.2 MAVN – MIPS Assembler Visokog Nivoa

MAVN (MIPS Assembler Visokog Nivoa) je alat koji prevodi program napisan na višem MIPS 32bit assemblerskom jeziku na osnovni assemblerski jezik. Viši MIPS 32bit assemblerski jezik uvodi koncept registarske promenljive, čime programeru omogućava da pri pisanju instrukcija koristi simbolička imena umesto stvarnih hardverskih resursa. Na taj način programer ne mora da prati koji su registri zauzeti niti koji je njihov sadržaj u svakom trenutku izvršavanja programa.

Radi jednostavnosti projekta, u instrukcijama se ne smeju koristiti pravi registarski resursi već isključivo deklarisanе promenljive.

1.3 Zadatak

U okviru projektnog zadatka realizovan je MAVN prevodilac koji prevodi programe pisane na višem assemblerskom jeziku na osnovni MIPS 32bit-ni assemblerski jezik. Prevodilac podržava detekciju leksičkih i sintaksnih grešaka uz generisanje odgovarajućih izveštaja. Izlaz iz prevodioca predstavlja assemblerski kod koji je moguće izvršiti na MIPS 32bit arhitekturi, tj. u QtSpim simulatoru.

Pored 10 osnovnih MAVN instrukcija definisanih specifikacijom, implementirane su i tri dodatne instrukcije:

- AND - logičko I (aritmetičko-logička operacija)
- SLT - Set Less Than (operacija poređenja)
- JR - Jump Register (operacija skoka)

2. Analiza problema

Osnovni problem koji prevodilac rešava jeste mapiranje neograničenog skupa registarskih promenljivih na ograničen skup realnih hardverskih registara. U našem slučaju u pitanju su četiri registra t0–t3 MIPS arhitekture.

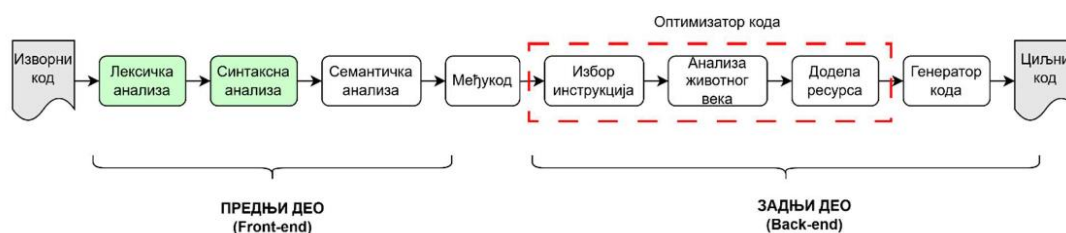
Kada broj registarskih promenljivih u programu prelazi broj raspoloživih registara, dolazi do konfliktnih situacija u kojima dve ili više promenljivih istovremeno zahtevaju isti registar. Rešavanje ovog problema zahteva analizu toka podataka i pametnu alokaciju resursa.

Konkretni izazovi koje je bilo potrebno rešiti:

- Leksička validacija ulaznog programa: prepoznavanje svih validnih tokena MAVN jezika i detekcija leksičkih grešaka.
- Sintaksna analiza: provera da program poštuje gramatiku MAVN jezika, uz proširenje za tri nove instrukcije.
- Analiza životnog veka promenljivih (*Liveness Analysis*): za svaku tačku programa odrediti koje promenljive su „žive“, tj. čija vrednost će biti korišćena u nastavku izvršavanja.
- Izgradnja grafa interferencije: dve promenljive su u smetnji ako im se životni vekovi preklapaju, tj. ako su žive u isto vreme.
- Alokacija registara bojenjem grafa: promenljivama koje nisu u smetnji može se dodeliti isti registar. Cilj je minimizovati broj korišćenih registara.
- Upravljanje prelivanjem (*spill*): kada nije moguće obojiti graf sa raspoloživim brojem boja (registara), prevodilac to detektuje i zaustavlja prevođenje.
- Generisanje izlaznog MIPS fajla: na osnovu izvršene alokacije generisati sintaksno ispravan .o fajl.

3. Koncept rešenja

Prevodilac je organizovan kao niz faza koje se izvršavaju sekvencijalno, pri čemu svaka faza priprema ulaz za narednu. Slika 1 prikazuje osnovne faze kompajlera (prevodioca).



[Slika 1 – Osnovne faze kompajlera (prevodioca)]

3.1 Leksička analiza

Leksička analiza je prva faza prevodioca. Njen zadatak je da ulazni tekst razloži na niz tokena i proveri da li su svi leksemi validni u MAVN jeziku. Implementirana je pomoću konačnog automata (FSM – *Finite State Machine*).

FSM je proširen za podršku tri nove instrukcije (AND, JR, SLT). Validacija formata deklaracija vrši se na dva mesta:

- `_mem` i `_reg` – validacija u FSM-u
- `_func` – validacija u lekseru (Do() metoda): ime funkcije mora početi malim slovom, a nastavak može biti kombinacija slova i brojeva, što odgovara opštem `T_ID` tokenu koji FSM već prepoznaje. Umesto pravljenja novog FSM stanja, Do() metoda ručno čita sledeći token i proverava da li je `T_ID`.

3.2 Sintaksna analiza

Sintaksna analiza proverava da li program poštuje gramatiku MAVN jezika. Parser prolazi kroz listu tokena i prijavljuje sintaksne greške.

Ključno proširenje u odnosu na početni kod sa vežbi: parser više ne „zaboravlja“ šta je pročitao. Za svaku prepoznatu instrukciju beleži:

- Koji registri se pišu (dst/def)
- Koji registri se čitaju (src/use)

Ove informacije se čuvaju u listama instrukcija i promenljivih, što je preduslov za korektnu analizu životnog veka.

Gramatika je proširena za tri nove instrukcije: logičko i, operaciju „manje od“ i безусловni skok na adresu iz registra.

Instrukcija	Sintaksa	Opis
AND	and rd, rs, rt	Logičko I: $rd = rs \& rt$
SLT	slt rd, rs, rt	$rd = 1$ ako je $rs < rt$, inače $rd = 0$
JR	jr rs	Bezuslovni skok na adresu iz registra rs

[Tabela 1 – Proširenje za MAVN assembler – 3 dodatne instrukcije]

3.3 Analiza životnog veka (Liveness Analysis)

Analiza životnog veka je analiza toka podataka koja za svaku tačku programa određuje da li je neka promenljiva „živa“, tj. da li će biti korišćena u nastavku izvršavanja. Ova faza direktno uslovljava alokaciju registara: ako dve promenljive nisu žive u isto vreme, mogu deliti isti registar.

Algoritam se odvija u tri koraka:

- Konstruisanje grafa toka upravljanja: za svaki čvor (instrukciju) definišu se skupovi prethodnika (pred), naslednika (succ), korišćenih (use) i definisanih (def) promenljivih.
- Iterativni algoritam: za svaku instrukciju računaju se skupovi out (promenljive žive na izlazu) i in (promenljive žive na ulazu). Iteracije se ponavljaju dok se skupovi budu isti u dve uzastopne iteracije.
- Rezultat: skupovi in i out za svaku instrukciju, koji opisuju koje promenljive su žive na ulasku i izlasku iz svakog čvora.

Posebna pažnja posvećena je instrukcijama grananja (B, BLTZ): za njih naslednik nije samo naredna instrukcija već i instrukcija na koju labela pokazuje, što je ispravno implementirano kroz sistem labela i funkciju koja konstruiše tok programa.

3.4 Zauzimanje resursa (Alokacija registara)

Alokacija registara vrši se bojenjem grafa interferencije. Algoritam se sastoji od četiri podfaze:

- Formiranje grafa interferencije: čvorovi su promenljive, a ivica između dve promenljive postoji ako su one žive u istom trenutku. Graf je predstavljen matricom interferencije.
- Uprošćavanje (Simplification): iterativno se određuje rang svakog čvora. Čvorovi čiji je rang manji od broja raspoloživih registara stavljaju se na stek, i uklanjaju se grane kojima je bio povezan sa drugim čvorovima.
- Prelivanje (Spill): ako ne postoji čvor sa dovoljno malim rangom, graf se ne može obojiti sa zadatim brojem registara. Prevodilac detektuje ovu situaciju i prekida prevođenje.
- Izbor boja (Coloring): čvorovi se vraćaju sa steka i dodeljuju im se boje (registri) koje nisu korišćene od strane suseda.

3.5 Generisanje MIPS fajla

Na osnovu prikupljenih informacija - memorijskih promenljivih, imena funkcije, instrukcija sa alociranim registrima i imena fajla - generiše se izlazni asemblerski .o fajl u skladu sa sintaksom MIPS 32bit asemblerskog jezika. Fajl sadrži:

- .globl direktivu sa imenom funkcije
- .data sekciju sa deklaracijama memorijskih promenljivih
- .text sekciju sa prevedenim instrukcijama i logičkim labelama

4. Programsko rešenje

4.1 Organizacija projekta

Projekat je nazvan MAVNTranslator. Svaka faza prevođenja organizovana je u poseban podfolder, čime je postignuta sledeća struktura:

```
MAVNTranslator/
├─ LexicalAnalysis/    # Leksička analiza (FSM)
├─ SyntaxAnalysis/     # Sintaksna analiza (Parser)
├─ LivenessAnalysis/   # Analiza životnog veka
├─ ResourceAllocation/ # Alokacija registara i graf interferencije
├─ MIPSGeneration/    # Generisanje asemblerske datoteke
└─ main.cpp           # Glavni program
```

4.2 Leksička analiza

Leksička analiza realizovana je kao konačni automat (FSM). Inicijalna verzija podržavala je osnovnih 10 MAVN instrukcija; u okviru projekta dodata je podrška za AND, JR i SLT.

Dodata su stanja u automatu sa konačnim brojem stanja - međustanja 21 i 22 do kojih se stiže kada FSM pročita samo `m` ili `r` iza donje crte (`_`). Tek kada sledi cifra, FSM prelazi u stanja koja emituju `T_M_ID` ili `T_R_ID`. Ako sledi slovo, FSM prelazi u stanje `T_ID` (nije validno ime za memorijsku ili registarsku promenljivu, ali jeste za standardni identifikator). Ako sledi `_`, ide u stanje greške `T_ERROR`.

Validacija imena funkcije (`_func`) implementirana je u `Do()` metodi leksera: token mora biti `T_ID`, što garantuje da ime počinje malim slovom (velika slova i cifre na početku daju `T_ERROR` odnosno `T_NUM`).

Implementirana su i dodatna stanja koja obezbeđuju ispravno tokenizovanje instrukcija koje su dodate kao proširenje za MAVN jezik.

4.3 Sintaksna analiza

Parser (klasa SyntaxAnalysis) je proširen u odnosu na početnu verziju sa vežbi. Dodata je podrška za tri nove instrukcije (AND, SLT, JR) u skladu sa gramatikom. Main je izmenjen - poziva se syntax.Do() za pokretanje sintaksne analize.

Parser (SyntaxAnalysis) je pre naših izmena, u obliku sa vežbi, samo proveravao sintaksu - prolazio kroz tokene, javljao greške, i zaboravljao sve. Dodato je da pamti šta je pročitao - za svaku instrukciju koju pronade, u sintaksoj analizi se sačuva koji registri se pišu (dst/def) i koji registri se čitaju (src/use). To se čuva u listama za instrukcije, odnosno promenljive.

Sintaksna analiza je implementirana korišćenjem pomoćnih funkcija:

findOrAddVariable(name, type) – traži promenljivu po imenu u listi; ako ne postoji, kreira je i dodaje (sprečava duplikate):

```
Variable* findOrAddVariable(const std::string& name, Variable::VariableType
type);
```

buildInstruction(type, dst, src) – kreira Instruction objekat i dodaje ga u listu (dst u m_def, src u m_use):

```
void buildInstruction(InstructionType type, Variables& dst, Variables& src);
```

buildControlFlow() – nakon što su sve instrukcije napravljene, prolazi kroz listu i povezuje ih (succ/pred). Prethodna instrukcija je pred trenutnoj, a sledeća je succ. Instrukcije grananja (B, BLTZ – *branch (less than zero)*) su izuzeci - naslenik je i instrukcija na koju pokazuje labela (tj. samo ta instrukcija, ukoliko je instrukcija B - *branch*):

```
void buildControlFlow();
```

4.4 Analiza životnog veka

Funkcija livenessAnalysis() vrši analizu unazad (od poslednje ka prvoj instrukciji). Za svaku instrukciju računa:

- out_new: unija in skupova svih njenih sledbenika (sukcesora).
- in_new: promenljive koje se koriste u instrukciji (use), kao i one koje su žive na izlazu (out), a nisu definisane (izmenjene) u toj instrukciji (def).

Iteracije se ponavljaju dok se out i in skupovi ne stabilizuju. Implementacija je prilagodjena zahtevima projekta na osnovu koda sa vežbi.

4.5 Alokacija resursa

Alokacija resursa implementirana je kroz sledeće funkcije:

- `doInterferenceGraph()` – Implementacija u prvom koraku prolazi kroz sve instrukcije i sakuplja sve jedinstvene registre iz svih skupova liveness analize (`m_use`, `m_def`, `m_in`, `m_out`), čime se garantuje da će svaka promenljiva dobiti svoj čvor u grafu i uspešno mapiran fizički registar. Svaka promenljiva koja se definiše u instrukciji (`m_def`) je u interferenciji sa svim promenljivama koje su u tom trenutku žive na izlazu (`m_out`). Time je sprečeno da se dodeli isti fizički registar promenljivama čiji se životni vekovi preklapaju, što je prethodno uzrokovalo greške u generisanju koda (pojavu nealociranih `??` simbola). Dimenzija matrice smetnji određena je na osnovu najveće pozicije registra (`maxPos+1`). Dve promenljive koje su istovremeno žive dobijaju oznaku `__INTERFERENCE__`.
- `doSimplification()` – računa stepen (broj smetnji) za svaki čvor. Iterativno sklanja čvor čiji je stepen manji od broja raspoloživih registara; ako takvog nema, vraća `NULL` (spill).
- `doResourceAllocation()` – ispravljen je kod sa vežbi: uklonjeno je dodeljivanje prve boje prvom čvoru; svim promenljivama se eksplicitno resetuje stanje na `__UNDEFINE__` pre bojenja. Dodat je `save.clear()` na početku kako bi se sprečilo nagomilavanje starih promenljivih kroz višestruka pokretanja.
- `getColor()` – kod sa vežbi je prilagođen specifikaciji projekta tako da podržava veći broj boja, tj. na osnovu grafa interferencije dodeljuje promenljivama neki od 4 registra.
- `freeInterferenceGraph()` – oslobadja matricu grafa iz memorije; destruktorka obilazi brisanje varijabli koje pripadaju instrukcijama.

4.6 Generisanje MIPS .o fajla

Faza generisanja koda koristi sve prethodno prikupljene informacije i generiše izlazni .mips fajl prema MIPS sintaksi. Struktura izlaznog fajla prikazana je na Tabeli 2:

```
.globl <ime_funkcije>

.data
<mem_promenljiva>: .word <vrednost>
...

.text
<ime_funkcije>:
    <instrukcija>    $t<x>, ...
    ...
```

[Tabela 2 –Struktura izlazne MIPS datoteke]

5. Uputstvo za pokretanje projekta

Za pokretanje projekta potrebno je dodati `.mavn` datoteku koju treba prevesti u podfolder *examples/* i u glavnoj datoteci projekta, u `main()` funkciji (`main.cpp` datoteka) navesti putanju do datoteke koja se prevodi (npr. `string fileName = "..\\..\\examples\\simple.mavn";`). Projekat se otvara i pokreće u okruženju *Visual Studio*, a Windows SDK verzija je 10.0. Potrebno je imati instaliran i namešten prevodilac i debager za programski jezik *C++*, a standard je ISO C++14.

Nakon pokretanja projekta, u terminalu će biti opisana greška pri prevođenju, ukoliko ima greške, kao i ispis svih faza prevođenja do tog momenta. Ukoliko nema greške, u terminalu će biti ispisan sadržaj prevedene datoteke, a ona sama će biti smeštena u istom folderu kao i ulazna datoteka samo sa ekstenzijom `.s` umesto `.mips`.

6. Verifikacija

Verifikacija je sprovedena na četiri testna primera koji pokrivaju različite aspekte prevodioca: osnovne instrukcije, složenu alokaciju sa prelivanjem, nove instrukcije i kompleksno grananje.

6.1 Testni skupovi

6.1.1 simple.mavn – Osnovne instrukcije

Testira ispravno prevođenje osnovnih instrukcija (la, lw, add) sa manjim brojem promenljivih (r1–r5). Očekivanje je da prevodilac alokira minimalan broj registara.

simple.mavn	simple.mips
<code>_mem m1 6;</code>	<code>.globl main</code>
<code>_mem m2 5;</code>	
	<code>.data</code>
<code>_reg r1;</code>	<code>m1: .word 6</code>
<code>_reg r2;</code>	<code>m2: .word 5</code>
<code>_reg r3;</code>	
<code>_reg r4;</code>	<code>.text</code>
<code>_reg r5;</code>	<code>main:</code>
	<code>la \$t0, m1</code>
<code>_func main;</code>	<code>lw \$t1, 0(\$t0)</code>

la r4, m1;	la \$t0, m2
lw r1, 0(r4);	lw \$t0, 0(\$t0)
la r5, m2;	add \$t0, \$t1, \$t0
lw r2, 0(r5);	
add r3, r1, r2;	

[Tabela 3 – Prevođenje datoteke simple.mavn]

Napomena: r2 i r3 dele isti registar (\$t0) jer im se životni vekovi ne preklapaju (r2 više nije živa kada r3 postaje živa). Ovo je optimalno rešenje sa samo 2 registra, potvrđeno izvršavanjem u QtSpim simulatoru.

6.1.2 multiply.mavn – Složena alokacija (spill i bez spilla)

Testira alokaciju sa većim brojem promenljivih (r1–r8) i labelama za grananje. Ovaj test je pokretan u dve varijante:

- Sa ograničenjem na 4 registra (uslov specifikacije): dolazi do preliivanja (spill) – prevodilac ispravno detektuje i signalizira da alokacija nije moguća.
- Sa ograničenjem na 5 registara: uspešno prevođenje, prikazano u nastavku.

multiply.mavn	multiply.mips (5 registara)
_mem m1 6; _mem m2 5; _mem m3 0;	.globl main
_reg r1; _reg r2; _reg r3; _reg r4;	
_reg r5; _reg r6; _reg r7; _reg r8;	.data
	m1: .word 6
_func main;	m2: .word 5
la r1, m1;	m3: .word 0
lw r2, 0(r1);	
la r3, m2;	.text
lw r4, 0(r3);	main:
li r5, 1;	la \$t0, m1
li r6, 0;	lw \$t4, 0(\$t0)
lab:	la \$t0, m2
add r6, r6, r2;	lw \$t3, 0(\$t0)
sub r7, r5, r4;	li \$t2, 1
addi r5, r5, 1;	li \$t1, 0

bltz r7, lab;	lab:
la r8, m3;	add \$t1, \$t1, \$t4
sw r6, 0(r8);	sub \$t0, \$t2, \$t3
nop;	addi \$t2, \$t2, 1
	bltz \$t0, lab
	la \$t0, m3
	sw \$t1, 0(\$t0)
	nop

[Tabela 4 – Prevođenje datoteke multiply.mavn]

6.1.3 commands.mavn – Nove instrukcije

Testira ispravnu podršku za tri dodate instrukcije: AND, SLT i JR. Program učitava dve vrednosti iz memorije, vrši logičko I, uređuje ih i skoče na adresu iz registra.

commands.mavn	commands.mips
_mem m1 6; _mem m2 5; _mem m3 10;	.globl main
_reg r1; _reg r2; _reg r3;	
_reg r4; _reg r5; _reg r6;	.data
	m1: .word 6
_func main;	m2: .word 5
la r4, m1;	m3: .word 10
lw r1, 0(r4);	
la r5, m2;	.text
lw r2, 0(r5);	main:
add r3, r1, r2;	la \$t2, m1
and r6, r1, r2;	lw \$t1, 0(\$t2)
slt r3, r1, r2;	la \$t0, m2
jr r4;	lw \$t0, 0(\$t0)
	add \$t0, \$t1, \$t0
	and \$t0, \$t1, \$t0
	slt \$t0, \$t1, \$t0

	jr \$t2
--	---------

[Tabela 5 – Prevođenje datoteke *commands.mavn*]

6.1.4 test.mavn – Kompleksno grananje i višestruke labele

Složan test koji proverava: učitavanje četiri memorijske promenljive (od m1 do m4), aritmetičke operacije, uslovni (bltz) i bezuslovni (b) skokovi na više labele (l_neg, l_pos, end), čuvanje rezultata i povratak pomoću JR. Sa 8 registarskih promenljivih i 4 raspoloživa registra, liveness analiza mora precizno da utvrdi koji se registri mogu deliti kako bi se izbeglo preliivanje.

test.mavn	test.mips
_mem m1 10; _mem m2 20;	.globl main
_mem m3 30; _mem m4 40;	
_reg r1; ... _reg r8;	.data
	m1: .word 10 m2: .word 20
_func main;	m3: .word 30 m4: .word 40
la r1, m1;	
lw r2, 0(r1);	.text
la r1, m2;	main:
lw r3, 0(r1);	la \$t3, m1
add r4, r2, r3;	lw \$t4, 0(\$t3)
la r1, m3;	la \$t3, m2
lw r5, 0(r1);	lw \$t2, 0(\$t3)
sub r6, r4, r5;	add \$t0, \$t4, \$t2
add r7, r6, r2;	la \$t3, m3
bltz r7, l_neg;	lw \$t1, 0(\$t3)
b l_pos;	sub \$t0, \$t0, \$t1
l_neg:	add \$t0, \$t0, \$t4
sub r8, r2, r3;	bltz \$t0, l_neg
b end;	b l_pos
l_pos:	l_neg:
add r8, r3, r5;	sub \$t0, \$t4, \$t2
end:	b end
sw r8, 0(r1);	l_pos:
nop;	add \$t0, \$t2, \$t1

j r r1;	end:
	sw \$t0, 0(\$t3)
	nop
	j r \$t3

[Tabela 6 – Prevođenje datoteke test.mavn]

8 registarskih promenljivih uspeo je da se alokira u samo 4 hardverska registra (što je minimalan broj po specifikaciji), zahvaljujući preciznoj liveness analizi koja je ispravno utvrdila da se životni vekovi većine promenljivih ne preklapaju.

6.2 Rezultati i analiza

Kroz verifikaciju utvrđeno je da su svi predviđeni testni primeri uspešno prevedeni. Svi generisani izlazi prošli su kroz validaciju u *QtSpim* simulatoru, gde je potvrđeno da semantika prevedenog koda u potpunosti odgovara izvornom ponašanju test-primeru.

Analizom generisanog koda uočeni su ključni aspekti performansi i funkcionalnosti sistema:

- Optimizacija alokacije registara: Algoritam za alokaciju registara pokazuje visok nivo efikasnosti. Prevodilac uspešno svodi broj korišćenih registara na neophodni minimum, što je direktno demonstrirano na primeru simple.mavn. U ovom testnom slučaju dobijeno je optimalnije rešenje u pogledu iskorišćenja resursa čak i u poređenju sa referentnim primerom iz same specifikacije projekta.
- Proširenje skupa instrukcija: Integracija novih instrukcija (AND, SLT, JR) uspešno je realizovana. Tokom testiranja potvrđeno je njihovo ispravno funkcionisanje i generisanje adekvatnog mašinskog koda u različitim scenarijima.
- Upravljanje resursima i detekcija preliivanja: Mehanizam za detekciju preliivanja registara (engl. *spill*) funkcioniše pouzdano. Ovo ponašanje je uspešno verifikovano kroz test-primer multiply.mavn, gde je sistem ispravno prepoznao preliivanje.

Konačni rezultati dovode do zaključka da implementirano rešenje zadovoljava sve zahteve specifikacije i u domenima optimizacije alokacije registara ostvaruje rezultate iznad inicijalno postavljenih referentnih vrednosti.

7. Zaključak

U okviru ovog projekta realizovan je funkcionalan MAVN prevodilac koji prevodi programe pisane na višem MIPS asemblerskom jeziku na osnovni MIPS 32bit asemblerski jezik. Prevodilac prolazi kroz sve klasične faze kompajlera: leksičku analizu, sintaksnu analizu, analizu životnog veka, alokaciju registara bojenjem grafa smetnji i generisanje izlaznog koda.

Postignuti rezultati pokazuju da rešenje:

- Ispravno prevodi sve testne primere, uključujući složene scenarije sa višestrukim grananjima i labelama.
- Ostvaruje optimalnu alokaciju registara — u pojedinim slučajevima bolje od primera navedenog u specifikaciji projekta, jer liveness analiza precizno identifikuje promenljive čiji se životni vekovi ne preklapaju i dozvoljava im deljenje istog registra.
- Uspešno detektuje situacije preliivanja (spill) kada broj promenljivih nadmašuje broj raspoloživih registara.
- Podržava tri dodatne instrukcije (AND, SLT, JR) van osnovnog skupa od 10 instrukcija.
- Generiše izlazne fajlove koji se mogu direktno izvršavati u QtSpim simulatoru.

Postoje i oblasti u kojima bi se rešenje moglo unaprediti u budućem radu. Implementacija podrške za preliivanje memorije kada se *spill* detektuje čime bi se automatski premeštale promenljive u memoriju kada dođe do preliivanja - bila bi najznačajnije unapređenje: time bi prevodilac mogao da obradi programe proizvoljne složenosti bez ograničenja na broj promenljivih. Pored toga, moguće je proširiti skup podržanih instrukcija dok se ne pokrije čitav MIPS32 skup, ili uvesti optimizacije na nivou koda (npr. eliminacija mrtvog koda). Uprkos tim mogućim proširenjima, postojeće rešenje u potpunosti ispunjava sve zahteve projektnog zadatka i demonstrira primenu klasičnih tehnika teorije prevođenja na praktičnom primeru.